



Università Politecnica delle Marche

Facoltà di Ingegneria

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica e
dell'Automazione

Implementazione di un Chatbot per Assistere gli Studenti in Ingresso

Docenti

Prof. Ursino Domenico
Dott. Buratti Christopher

Componenti del gruppo

Dott. Tempera Fabio
Dott. Marcianesi Luca
Dott. Vianello Gabriele

ANNO ACCADEMICO 2024-2025

Indice

1	Introduzione	4
1.1	Requisiti Tecnici e Funzionali	4
1.2	Tecnologie, Framework e Strumenti Utilizzati	5
1.2.1	RASA	5
1.2.2	Docker	6
1.2.3	Telegram API e NGROK	7
1.2.4	Ambiente di Sviluppo	7
2	Architettura della soluzione	8
2.1	RASA	9
2.1.1	File di configurazione principale: config.yml	11
2.1.2	Dominio del bot: domain.yml	12
2.1.3	Dati di training: dataset sintetico manuale	13
2.2	Custom Actions ed Estensioni Funzionali	14
2.2.1	Gestione dei processi e notifiche	14
2.2.2	Recupero dinamico delle informazioni (RAG e LLM)	15
2.2.3	Gestione strutturata dell'offerta formativa via Data- base	16
2.3	Integrazione con sistemi di messaggistica	18
3	Uncounsciously Sincere Bot	19
3.1	Esempi di flusso	19
3.1.1	Inizio e Fine Chat	19
3.1.2	Challenge: "Sei un bot?"	20
3.1.3	FAQ: Scadenze, Costi e Requisiti	21
3.1.4	Form di Iscrizione (enrollment_form)	24
3.1.5	Error Recovery e Fallback	29
3.2	Vantaggi e Limitazioni del Sistema	31
3.2.1	Vantaggi Principali	31
3.2.2	Limitazioni Attuali e Possibili Miglioramenti	32
3.2.3	Possibili Estensioni Future	32

4	Esecuzione del chatbot	34
4.1	Setup iniziale e installazione	34
4.1.1	Requisiti di sistema	34
4.1.2	Clonazione e configurazione base	34
4.2	Esecuzione tramite Docker Compose	35
4.2.1	Configurazione del file docker-compose	35
4.2.2	Costruzione dell'immagine e avvio	36
4.2.3	Recupero dell'indirizzo Ngrok	36
4.3	Esecuzione in ambiente locale alternativo	36
4.3.1	Installazione delle dipendenze	37
4.3.2	Training ed esecuzione manuale	37
4.4	Configurazione delle email	37
4.4.1	Configurazione per Gmail	37
4.4.2	Variabili d'ambiente	38
4.5	Integrazione con Telegram	38
4.5.1	Creazione del bot e token	38
4.5.2	Modifica delle credenziali	38
4.6	Risoluzione dei problemi	39
4.6.1	Mancata risposta su Telegram	39
4.6.2	Porta 5005 già in uso	39
4.6.3	Errori di memoria (OOM)	39
4.7	Considerazioni per la produzione	39

Elenco delle figure

2.1	Architettura del modello.	8
2.2	Logo di Rasa.	9
3.1	Esempio di apertura conversazione con saluto.	20
3.2	Esempio di chiusura conversazione con messaggio di arri- vederci.	20
3.3	Esempio di risposta al bot challenge ("Sei un bot?").	21
3.4	Esempio di richiesta informazioni sulle scadenze di iscrizione.	22
3.5	Esempio di richiesta informazioni sui costi universitari.	23
3.6	Esempio di raccolta del nome studente nel form di iscrizione.	25
3.7	Esempio di selezione del tipo di laurea nel form di iscrizione.	26
3.8	Esempio di selezione del corso di studio nel form di iscrizione.	27
3.9	Esempio di completamento e sottomissione del form di iscri- zione con conferma personalizzata.	28
3.10	Email inviata dal bot.	29
3.11	Esempio di attivazione del FallbackClassifier per richiesta non riconosciuta.	30
3.12	Esempio di gestione di domande fuori ambito con risposta appropriata.	31

1 Introduzione

L'avanzamento delle tecnologie di Intelligenza Artificiale e dell'Hardware a loro supporto negli ultimi anni, ha introdotto innovazioni e miglioramenti significativi nei ChatBot.

In contesti operativi come segreterie, uffici informativi o amministrativi sommersi da richieste di ottenere maggiori informazioni o solo chiarimenti su pratiche burocratiche, i chatbot sono un "*game changer*": la totale automazione di processi come quelli descritti può ridurre notevolmente il peso che grava sul personale.

L'obiettivo che si pone questo progetto è quello di fornire una solida base per sviluppare ChatBot conversazionali intelligenti. Il sistema sarà focalizzato sull'assistenza all'iscrizione ad un corso universitario, con funzionalità di emailing; il tutto in modalità bilingue, inglese e italiano. In questo modo si fornirà maggiore supporto agli studenti erasmus in entrata o comunque provenienti da contesti internazionali.

Il sistema realizzato non sostituirà il lavoro umano, dove i dipendenti potranno concentrarsi su compiti specifici o ad un maggiore livello di astrazione; inoltre, le risposte date dal ChatBot agli utenti permetterà loro di avere maggiori informazioni nell'eventualità in cui dovessero comunque recarsi presso - ad esempio - la segreteria, facilitando non poco il lavoro del personale.

1.1 Requisiti Tecnici e Funzionali

Attraverso un'attenta analisi dei requisiti e delle finalità applicative del sistema, si è deciso di realizzare una soluzione software che utilizzi un ChatBot con cui sarà possibile interagire via social, per essere più vicini all'utente finale. E' importante sottolineare che l'architettura realizzata si pone come obiettivo, di riuscire a migliorare significativamente l'esperienza dei potenziali studenti, rendendo il processo di iscrizione più accessibile, trasparente e user-centric.

La soluzione realizzata dovrà rispettare i seguenti requisiti tecnici e funzionali:

1. **Multi piattaforma:** per garantire la massima flessibilità sia agli sviluppatori che agli utilizzatori finali, senza doversi preoccupare di costruire una piattaforma ad-hoc.
2. **Modularità:** per poter configurare il software in base alle necessità di utilizzo, come le azioni avanzate in funzioni delle domande/risposte dell'utente.
3. **Supporto multilingue:** per offrire maggiore assistenza soprattutto a studenti provenienti dall'estero, così da alleggerire il carico sugli uffici preposti.
4. **Scalabilità:** per aumentarne la precisione o rispondere meglio a nuove esigenze si dovrà poter riaddestrare facilmente il modulo che si occuperà di rispondere all'utente e analizzarne le domande.
5. **Inoltro Email:** come funzionalità avanzata di base si vuole che il sistema realizzato sia in grado di gestire l'inoltro di email attraverso apposita API, e.g. *Google Gmail*.
6. **Sicurezza:** poichè il sistema dovrà gestire nomi di studenti e indirizzi email, si vuole garantire la massima protezione dei dati.
7. **Personalizzazione:** avere un architettura modulare e altamente personalizzabile consentirà un più facile riadattamento del sistema a nuovi contesti operativi.

1.2 Tecnologie, Framework e Strumenti Utilizzati

Tutto lo stack tecnologico che sarà approfondito in questa sezione è basato su **Python** linguaggio estremamente versatile ed espressivo, permette di realizzare applicativi potenti e modulari grazie alle sue numerose librerie. In questo progetto è stata utilizzata la versione 3.10 scelta per la sua stabilità e compatibilità con gli altri strumenti e librerie utilizzate nella soluzione.

1.2.1 RASA

Il cuore dell'architettura progettata è il chatbot, sviluppato con il framework **RASA 3.6**, leader nel settore dello sviluppo di assistenti conversazionali; La scelta di RASA è motivata da diverse caratteristiche, discusse di seguito:

- **Open Source e On-Premise:** a differenza di soluzioni cloud-based come Dialogflow o Lex, Rasa permette di mantenere il pieno controllo sui dati e sull'infrastruttura, aspetto cruciale quando si gestiscono informazioni sensibili degli studenti, che saranno conservate per il solo tempo di interazione con il ChatBot.
- **Machine Learning Avanzato:** Rasa integra algoritmi di machine learning all'avanguardia per la comprensione del linguaggio naturale (NLU) e la gestione del dialogo, permettendo conversazioni più naturali e adattive rispetto a chatbot basati su regole statiche.
- **Flessibilità e Personalizzazione:** il framework offre totale libertà nella configurazione della pipeline NLP, delle policy conversazionali e delle custom actions, permettendo di adattare il bot a requisiti specifici del dominio universitario.

Il sistema implementato si articola in diversi componenti chiave che collaborano per offrire un'esperienza conversazionale fluida:

- **Rasa NLU (Natural Language Understanding):** si occupa dell'interpretazione delle richieste degli utenti, identificando intenti ed estraendo entità rilevanti (nomi, corsi, tipi di laurea).
- **Rasa Core:** gestisce il flusso del dialogo, decidendo come rispondere in base al contesto della conversazione. Il sistema utilizza un approccio ibrido che combina regole deterministiche per comportamenti critici e machine learning per situazioni più complesse o impreviste.
- **Form Conversazionale:** per la raccolta strutturata delle informazioni di iscrizione (nome, tipo di laurea, corso), il bot utilizza il meccanismo di form di Rasa, che permette di guidare l'utente attraverso un processo passo-passo mantenendo la naturalezza della conversazione.
- **Gestione degli Errori:** il sistema monitora costantemente la confidenza delle predizioni e attiva meccanismi di recovery quando necessario per chiedere chiarimenti all'utente.

1.2.2 Docker

Per rispettare il requisito della capacità di operare su diverse piattaforme hardware e software, è stato scelto **Docker**: un potente motore di containerizzazione che permette di replicare l'ambiente di sviluppo in produzione, garantendo l'isolamento delle applicazioni ed una maggiore scalabilità.

L'utilizzo di questa tecnologia ha permesso implementazione e testing con assoluta efficienza, andando a risolvere problematiche di incompatibilità nelle dipendenze tra i diversi moduli software.

1.2.3 Telegram API e NGROK

Il bot è progettato per operare su **Telegram**, sfruttando la popolarità e la diffusione della piattaforma tra i giovani utenti.

Ngrok: per esporre il bot locale a Internet tramite un tunnel HTTPS sicuro, necessario per ricevere i webhook da Telegram.

Webhook: Telegram comunica con Rasa tramite webhook, garantendo risposte in tempo reale senza necessità di polling continuo.

1.2.4 Ambiente di Sviluppo

Per sviluppare il codice in modo efficace e flessibile è stato utilizzato Visual Studio Code, con Git/GitHub per il versionamento della soluzione, che è attualmente disponibile nella repository del progetto.

Questo approccio tecnico garantisce un sistema robusto, scalabile e facilmente manutenibile, capace di gestire conversazioni complesse pur mantenendo un'interfaccia semplice e intuitiva per l'utente finale.

2 Architettura della soluzione

descrizione iniziale della nuova architettura e nuove feature del progetto

La soluzione proposta si articola su tre moduli distinti che collaborano per garantire un'esperienza utente fluida e coerente. L'architettura è stata progettata per essere modulare, scalabile e facilmente manutenibile, separando nettamente la logica di comprensione del linguaggio, la gestione del dialogo e l'esecuzione delle azioni esterne.

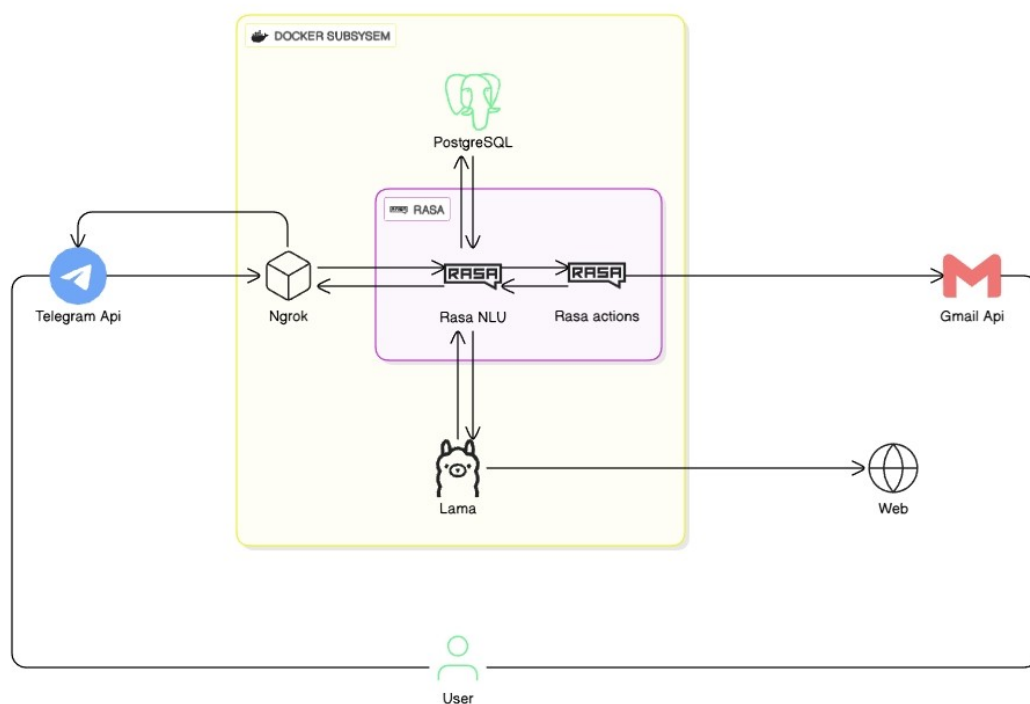


Figura 2.1: Architettura del modello.

2.1 RASA



Figura 2.2: Logo di Rasa.

Rasa è un framework open-source di livello industriale progettato per lo sviluppo di chatbot e assistenti virtuali contestuali. A differenza di molte soluzioni cloud-based che operano come "black box", Rasa offre un controllo totale sull'infrastruttura e sui dati, permettendo di creare agenti conversazionali altamente personalizzabili. Questa caratteristica è fondamentale per gestire conversazioni complesse che non seguono un albero decisionale rigido, ma si adattano dinamicamente agli input dell'utente grazie all'integrazione di moduli avanzati per l'elaborazione del linguaggio naturale e la gestione probabilistica del dialogo.

Nel progetto "Uncounsciously Sincere Bot", è stata impiegata la versione 3.6.13 di Rasa. La scelta di questa specifica versione è dettata dalla sua comprovata stabilità e dalla ricchezza di funzionalità necessarie per gestire gli scenari articolati tipici del contesto di ammissione universitaria, dove le domande possono variare da semplici richieste informative a procedure burocratiche complesse.

Il framework si divide in due macro-componenti principali: Rasa NLU e Rasa Core.

Rasa NLU (Natural Language Understanding) è il componente responsabile dell'interpretazione semantica delle richieste degli utenti. Questo modulo trasforma il testo libero, spesso destrutturato e rumoroso, in dati strutturati comprensibili dalla macchina. Le sue funzioni principali includono:

- Identificare l'intento dell'utente (intent classification), ovvero comprendere cosa l'utente vuole ottenere, come chiedere le scadenze di iscrizione, conoscere i costi delle tasse o richiedere informazioni specifiche su un corso di laurea.
- Estrarre entità rilevanti dal testo (entity extraction), isolando i dettagli chiave necessari per soddisfare la richiesta, come il nome dello studente, il tipo di laurea desiderato o il programma di studio specifico.

- Analizzare il linguaggio naturale in contesti multilingua, supportando efficacemente sia l'italiano che l'inglese per garantire l'accessibilità a studenti internazionali.

Nel progetto, la pipeline NLU è stata configurata con una serie di componenti avanzati che lavorano in sequenza:

- **WhitespaceTokenizer:** esegue la tokenizzazione iniziale dividendo il testo in base agli spazi bianchi. È una scelta efficiente per le lingue occidentali come l'italiano e l'inglese che utilizzano separatori chiari.
- **RegexFeaturizer:** arricchisce la rappresentazione del testo estraendo pattern basati su espressioni regolari. Questo componente è particolarmente utile per riconoscere entità con formati standardizzati, come codici fiscali, numeri di matricola o formati email.
- **CountVectorsFeaturizer:** si occupa dell'estrazione delle feature testuali. Nel modello ne vengono utilizzate due istanze: una a livello di parola e una a livello di caratteri (n-grams da 1 a 4). L'analisi per caratteri è cruciale per rendere il bot robusto contro gli errori di battitura, permettendo al modello di riconoscere parole anche se scritte in modo parzialmente errato.
- **DIETClassifier:** il cuore della pipeline, un'architettura a trasformatore (Dual Intent Entity Transformer) che gestisce contemporaneamente la classificazione degli intenti e l'estrazione delle entità. Addestrato su 100 epoche, questo componente apprende le relazioni complesse tra le parole e il contesto della frase.
- **EntitySynonymMapper:** normalizza i valori delle entità mappando diversi sinonimi a un unico valore canonico (ad esempio, mappando "CS", "Informatica" e "Computer Science" allo stesso identificativo di corso), semplificando la logica di backend.
- **ResponseSelector:** un modello specializzato per la gestione delle risposte "chitchat" e FAQ, che seleziona la replica più appropriata tra un insieme di candidati predefiniti.
- **FallbackClassifier:** funge da rete di sicurezza gestendo le richieste ambigue. Configurato con una soglia di confidenza del 30%, intercetta i messaggi che il modello non è riuscito a interpretare con certezza, attivando comportamenti di fallback eleganti invece di fornire risposte errate.

Rasa Core è il motore decisionale che gestisce il flusso del dialogo. A differenza dei vecchi chatbot basati su regole rigide, Rasa Core utilizza il machine learning per prevedere l'azione successiva più appropriata in

base al contesto attuale e alla storia della conversazione. Questo modulo utilizza:

- Algoritmi di machine learning (in particolare la TED Policy) per apprendere dai dati di esempio e generalizzare su conversazioni mai viste prima.
- Le "stories", ovvero scenari di addestramento che descrivono possibili interazioni tra utente e bot (come il percorso di iscrizione, la ricerca di informazioni o il cambio lingua).
- Le "rules", che definiscono comportamenti deterministici per situazioni che devono seguire sempre lo stesso schema logico (ad esempio, la risposta alla domanda "sei un bot?" o la gestione degli errori).
- I "forms", strumenti potenti per la raccolta strutturata di informazioni, essenziali per guidare l'utente attraverso processi complessi come la compilazione di una domanda di ammissione.

Grazie a questi componenti, Rasa permette di costruire un assistente che non si limita a rispondere a singole domande (modalità "botta e risposta"), ma è in grado di sostenere una conversazione coerente, mantenendo il contesto per più turni di dialogo.

L'implementazione concreta di Rasa nel progetto richiede la configurazione di diversi file chiave che definiscono il cervello e il comportamento del bot. Questa sezione descrive come i componenti teorici sono stati tradotti in una configurazione pratica per l'assistente universitario.

2.1.1 File di configurazione principale: config.yml

Il file `config.yml` è il punto centrale dove vengono definiti gli iperparametri del modello, la pipeline NLU e le policy di gestione del dialogo.

Pipeline NLU:

```
pipeline:
- name: WhitespaceTokenizer
- name: RegexFeaturizer
- name: CountVectorsFeaturizer
  analyzer: word
- name: CountVectorsFeaturizer
  analyzer: char_wb
  min_ngram: 1
  max_ngram: 4
- name: DIETClassifier
```

```

    epochs: 100
    constrain_similarities: true
- name: EntitySynonymMapper
- name: ResponseSelector
  epochs: 100
- name: FallbackClassifier
  threshold: 0.3
  ambiguity_threshold: 0.1

```

La configurazione scelta mira a massimizzare la comprensione in un contesto dominio-specifico. L'uso di `char_wb` (character word boundaries) nel featurizer è stato determinante per gestire la variabilità con cui gli studenti possono digitare i nomi dei corsi o le domande, rendendo il sistema resiliente ai typo. Il parametro `constrain_similarities: true` nel classificatore DIET aiuta inoltre a calibrare meglio i punteggi di confidenza, evitando falsi positivi su intenti simili.

2.1.2 Dominio del bot: `domain.yml`

Il file `domain.yml` rappresenta la "conoscenza del mondo" del bot, elencando tutto ciò che l'assistente può capire, dire o ricordare.

Intenti (25+): Gli intenti coprono l'intero spettro delle necessità di una matricola:

- Interazione sociale: `greet`, `goodbye`.
- Recupero informazioni: `ask_deadlines`, `ask_costs`, `ask_requirements`.
- Operatività: `apply_for_course`.
- Meta-comunicazione: `bot_challenge`, `language_italian`, `language_english`.
- Gestione errori: `nlu_fallback`, `out_of_scope`.

Entità e Slot: Le entità rappresentano i dati estratti, mentre gli slot sono la memoria a lungo termine del bot durante la sessione. Sono stati definiti slot specifici per mantenere lo stato della compilazione:

- `student_name`, `degree_type`, `course_name`, `email`: slot necessari per completare l'iscrizione.

Form di iscrizione: Il meccanismo dei form è gestito direttamente nel dominio per vincolare l'utente a fornire tutte le informazioni necessarie prima di procedere.

```

forms:
  enrollment_form:

```

```
required_slots:
  - student_name
  - degree_type
  - course_name
  - email
```

Grazie agli slot mapping, il bot è in grado di riempire questi campi sia se l'utente li fornisce spontaneamente in una frase ("Voglio iscrivermi a Informatica, mi chiamo Mario"), sia se vengono richiesti esplicitamente dal bot uno alla volta.

Registro delle Azioni: infine, il file di dominio agisce come registro ufficiale per le funzionalità custom implementate in Python. Per abilitare l'esecuzione del codice esterno, sono state dichiarate le seguenti azioni:

- `action_send_email` e `action_send_enrollment_email`: per gestire la comunicazione SMTP verso gli studenti.
- `action_get_university_info`: il punto di ingresso per il modulo ibrido di scraping e AI generativa (RAG), che permette al bot di recuperare informazioni non presenti staticamente nel dominio.

2.1.3 Dati di training: dataset sintetico manuale

Un aspetto distintivo e fondamentale di questo progetto riguarda l'origine dei dati di addestramento. Non esistendo un corpus preesistente di conversazioni reali relative alle specifiche procedure di questa università, l'intero dataset utilizzato per addestrare i file `nlu.yml`, `stories.yml` e `rules.yml` è stato creato manualmente.

Si tratta di un **dataset sintetico artigianale**: ogni singola frase di esempio, ogni variazione linguistica, ogni sinonimo e ogni possibile percorso narrativo nelle storie è stato ideato, scritto e validato dagli sviluppatori. Questo lavoro di data engineering ha richiesto di:

- Simulare centinaia di modi diversi in cui uno studente potrebbe porre la stessa domanda (es. "quanto costa?", "tasse da pagare", "prezzo iscrizione").
- Creare esempi bilanciati per la lingua italiana e inglese per evitare bias verso una delle due lingue.
- Anticipare i possibili errori o le risposte parziali degli utenti per costruire storie robuste.

Sebbene laborioso, l'uso di un dataset sintetico creato a mano garantisce che i dati siano perfettamente puliti e allineati con gli obiettivi del dominio, eliminando il rumore tipico dei dataset generici scaricati dal web.

Esempio di NLU Training Data (data/nlu.yml):

- intent: ask_deadlines
- examples:
 - Quando scade il bando?
 - Qual è la scadenza di iscrizione?
 - When is the deadline?
 - What's the application deadline?
 - Entro quando devo pagare?

Esempio di Rules (data/rules.yml): Le regole assicurano che, una volta attivato l'intento di iscrizione, il controllo passi invariabilmente al form dedicato:

- rule: Attiva form di iscrizione
- steps:
 - intent: apply_for_course
 - action: enrollment_form
 - active_loop: enrollment_form

2.2 Custom Actions ed Estensioni Funzionali

La logica applicativa complessa, che non può essere gestita dal semplice flusso di dialogo predefinito, risiede nel file `actions/actions.py`. Questo componente agisce come un server indipendente (Action Server) che esegue codice Python arbitrario in risposta agli intenti predetti dal modello NLU.

L'implementazione sviluppata estende le capacità native di Rasa integrando due tipologie di azioni: quelle transazionali, per la gestione dei processi burocratici, e quelle informative dinamiche, che sfruttano tecniche di Generative AI per il recupero di informazioni in tempo reale.

2.2.1 Gestione dei processi e notifiche

La prima categoria di azioni riguarda la chiusura dei cicli operativi, come l'iscrizione a un corso. Le classi `ActionSendEmail` e `ActionSendEnrollmentEmail` gestiscono la comunicazione asincrona con l'utente.

Il flusso logico implementato prevede:

1. Il recupero dei valori dagli slot riempiti durante la conversazione (nome, corso, email, tipologia di laurea).
2. La connessione sicura a un server SMTP (configurato via variabili d'ambiente per garantire la sicurezza delle credenziali).
3. La selezione dinamica del protocollo di sicurezza (SSL sulla porta 465 o TLS sulla porta 587) in base all'ambiente di deployment.
4. L'invio di un messaggio formattato che conferma l'avvenuta ricezione dei dati, offrendo un feedback immediato allo studente.

2.2.2 Recupero dinamico delle informazioni (RAG e LLM)

La vera innovazione architetturale del progetto risiede nella classe `ActionGetUniversityInfo`. A differenza dei chatbot tradizionali, limitati alle risposte statiche ("hard-coded") nel database, il sistema implementa un approccio ibrido che combina il web scraping in tempo reale con la generazione di testo tramite LLM (Large Language Model).

Questa azione viene invocata quando l'utente richiede informazioni specifiche (es. "tasse", "alloggi", "corsi") che potrebbero cambiare nel tempo o che sono troppo dettagliate per essere mantenute nel file di dominio. Il funzionamento si basa sull'orchestrazione di due librerie avanzate:

- **Crawl4AI**: una libreria di web crawling asincrono progettata per l'estrazione efficiente di contenuti ottimizzati per l'AI. Il sistema mappa l'argomento richiesto dall'utente a URL specifici del portale di ateneo (es. `univpm.it/Entra/Tasse_e_contributi`). Una volta scaricata la pagina, il contenuto viene convertito in formato Markdown e pulito tramite espressioni regolari (Regex) per rimuovere elementi non semantici come menu di navigazione, immagini, footer e script, riducendo drasticamente il rumore e il consumo di token.
- **Ollama (Local LLM Inference)**: il testo estratto viene inviato come contesto a un'istanza locale di Ollama, che esegue il modello `qwen3:0.6B`. Questo modello, scelto per il bilanciamento tra velocità di inferenza e accuratezza, riceve un prompt strutturato secondo il paradigma RAG (Retrieval-Augmented Generation) *on-the-fly*. E' importante sottolineare che i modelli della famiglia `qwen3` sono *Reasoning*, architettura che permette di raggiungere ottime prestazioni a costi computazionali decisamente inferiori.

Il flusso è: User Query → Crawl4AI (Retrieval) → Text Cleaning → Context Injection (Augmentation) → Qwen Inference (Generation) → Risposta

Il prompt ingegnerizzato segue lo schema:

“Rispondi alla domanda dell’utente usando SOLO il contesto fornito di seguito. { context }”

Questo vincolo riduce le “allucinazioni” tipiche dei modelli generativi, costringendo l’AI a basarsi esclusivamente sui dati ufficiali appena scaricati dal sito dell’università.

L’architettura risultante permette al bot di rispondere a domande su dati volatili (come scadenze aggiornate o importi delle tasse) senza necessitare di un continuo riaddestramento del modello NLU, delegando la comprensione del contenuto non strutturato al modello generativo.

2.2.3 Gestione strutturata dell’offerta formativa via Database

Parallelamente al modulo RAG, utilizzato per il recupero di informazioni discorsive e mutevoli, è stata implementata un’architettura di persistenza dati relazionale per gestire l’offerta formativa ufficiale dell’ateneo.

Mentre l’approccio generativo descritto in precedenza è ideale per il testo libero, la necessità di fornire agli studenti l’elenco esatto degli esami o i dettagli specifici dei corsi di laurea richiede una precisione assoluta. L’uso di un database SQL dedicato permette di adottare un approccio deterministico, eliminando il rischio di “allucinazioni” tipiche dei Large Language Models quando devono elencare dati fattuali rigidi.

Schema del Database

Il database è stato modellato attorno a due entità principali, collegate da una relazione uno-a-molti che rispecchia la struttura accademica:

- **Tabella** `degree`: funge da catalogo dei corsi di laurea. Ogni record include un identificativo univoco (es. “L8”), il nome del corso, la tipologia (Triennale, Magistrale, Ciclo Unico) e l’area tematica (Ingegneria, Economia, Medicina, ecc.).
- **Tabella** `course`: contiene i singoli insegnamenti (esami). Ogni esame è collegato al corso di laurea di appartenenza tramite una chiave esterna (`degree_id`) e possiede un attributo booleano `is_mandatory` che distingue gli esami fondamentali da quelli a scelta.

Di seguito è riportato un estratto significativo dello script SQL utilizzato per il popolamento, che evidenzia la categorizzazione dei dati e le relazioni tra le tabelle:

```
-- Definizione dei corsi di laurea
INSERT INTO degree (id, name, type, category) VALUES
('L8', 'Ingegneria Informatica...', 'Bachelor's Degree',
```

```

'Engineering'),
('LM32', 'Ingegneria Informatica...', 'Master's Degree',
'Engineering'),
('LM41', 'Medicina e Chirurgia', 'Single-Cycle Degree',
'Medicine');

-- Associazione degli insegnamenti (Foreign Key su degree_id)
INSERT INTO course (degree_id, name, is_mandatory) VALUES
-- Esami obbligatori per L8
('L8', 'Programming and Algorithms', TRUE),
('L8', 'Calculus and Analytical Geometry', TRUE),
-- Esami a scelta per L8
('L8', 'Advanced Robotics', FALSE),
('L8', 'Renewable Energy Systems', FALSE),
-- Esami magistrali per LM32
('LM32', 'Artificial Intelligence', TRUE),
('LM32', 'Cybersecurity', TRUE);

```

Integrazione con Rasa

Per interrogare questi dati strutturati, è stata sviluppata una Custom Action specifica nel backend Python. Il flusso operativo che si attiva quando uno studente chiede "Quali esami ci sono a Ingegneria Informatica?" è il seguente:

1. Il modello NLU estrae l'entità relativa al corso di interesse (es. `degree_name: "Ingegneria Informatica"`).
2. L'azione esegue una query SQL parametrica verso il database. L'uso di query parametriche è essenziale per prevenire vulnerabilità di tipo SQL Injection.
3. Il sistema recupera prima l'ID del corso corrispondente dalla tabella `degree` e successivamente esegue una JOIN con la tabella `course` per ottenere la lista degli insegnamenti.
4. I dati grezzi estratti dal database vengono elaborati e formattati in un elenco testuale leggibile, separando visivamente gli esami obbligatori da quelli opzionali, per poi essere inviati come risposta all'utente.

Questa soluzione ibrida conferisce all'architettura una doppia natura: la flessibilità linguistica dell'LLM per la comprensione delle intenzioni e la rigosità di un sistema gestionale classico per l'erogazione dei dati istituzionali.

2.3 Integrazione con sistemi di messaggistica

L'ultimo tassello dell'architettura è l'interfaccia verso il mondo esterno. Il file `credentials.yml` gestisce i token di autenticazione per le piattaforme supportate. Nel caso di Telegram, la configurazione collega il bot all'API pubblica tramite un webhook sicuro:

```
telegram:
  access_token: "YOUR_TELEGRAM_BOT_TOKEN"
  verify: "YOUR_BOT_USERNAME"
  webhook_url: "https://your-ngrok-url.ngrok.io/
               webhooks/telegram/webhook"
```

Parallelamente, il file `endpoints.yml` istruisce Rasa su dove trovare i servizi ausiliari, come l'action server (tipicamente su `localhost:5055`) e il tracker store, che nel setup attuale utilizza la memoria RAM per velocità, ma che è predisposto per essere migrato su Redis o SQL in un ambiente di produzione.

Questa configurazione architetturale fornisce una base solida: unisce la flessibilità del machine learning per la comprensione del linguaggio alla robustezza delle regole deterministiche per i processi critici, il tutto alimentato da un dataset curato manualmente per garantire la massima pertinenza nel contesto universitario.

3 Uncsciously Sincere Bot

3.1 Esempi di flusso

In questo capitolo verranno mostrati esempi d'uso del bot tramite flussi che vanno dal semplice saluto alla richiesta di informazioni per l'iscrizione, con conseguente invio di email per confermare le procedure effettuate.

3.1.1 Inizio e Fine Chat

Come per qualsiasi chatbot, il bot fornisce un saluto iniziale quando l'utente lo contatta e un messaggio di arrivederci al termine della conversazione.

L'intent `greet` attiva una risposta di benvenuto in inglese che introduce il bot come assistente per le ammissioni universitarie. L'intent `goodbye` attiva una risposta di chiusura con auguri di successo all'utente.

Questi messaggi sono fondamentali per creare un'esperienza user-friendly e professionale.

Un esempio di inizio e fine conversazione è riportato nelle seguenti figure, dove l'utente saluta il bot e riceve una risposta di benvenuto appropriata; allo stesso modo l'utente dice arrivederci e il bot fornisce un messaggio di congedo professionale.

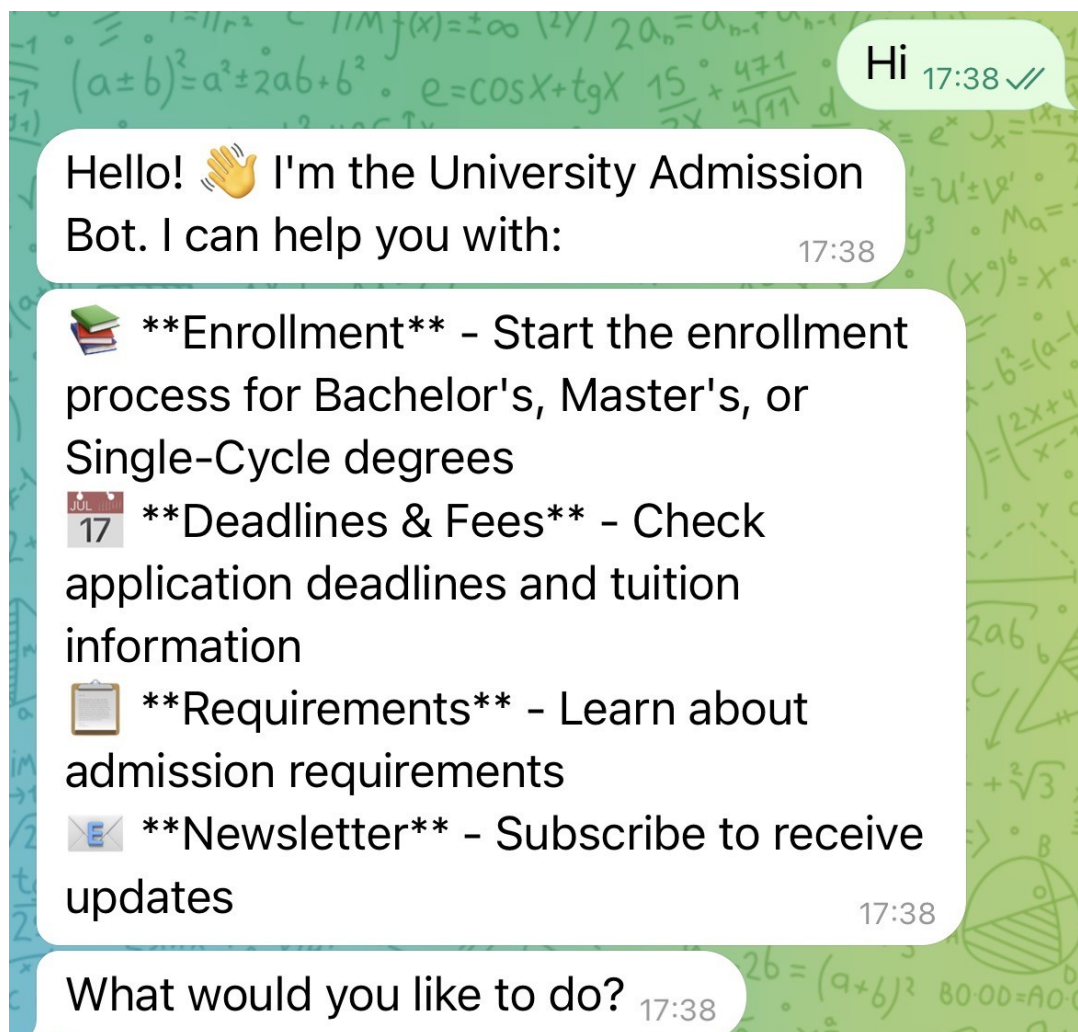


Figura 3.1: Esempio di apertura conversazione con saluto.

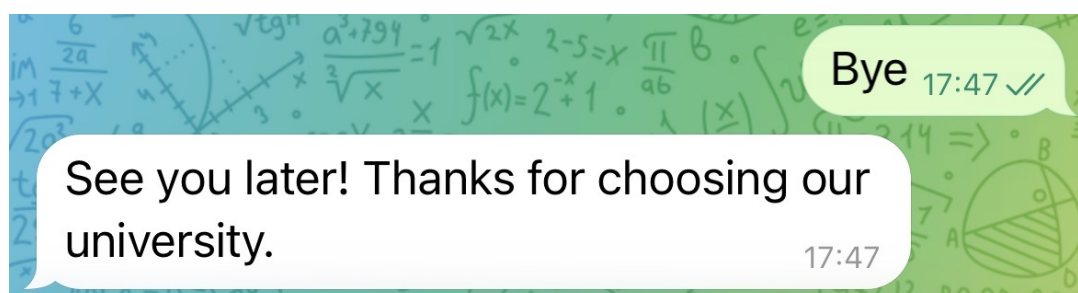


Figura 3.2: Esempio di chiusura conversazione con messaggio di arrivederci.

3.1.2 Challenge: "Sei un bot?"

Quando l'utente chiede "Sei un bot?" o domande simili, l'intent `bot_challenge` viene attivato. Il bot risponde in modo affermativo e trasparente, dichiarando di essere un agente conversazionale basato su Rasa, disponibile per aiutare con informazioni sull'ammissione universitaria.

Questa funzionalità è importante per:

- Trasparenza: l'utente sa che sta interagendo con un bot, non con una persona.
- Fiducia: una risposta onesta e diretta rafforza la credibilità del servizio.
- Gestione delle aspettative: l'utente comprende i limiti e le capacità del bot.

Un esempio di bot challenge è riportato di seguito, dove l'utente chiede "Sei un bot?" e il bot risponde in modo trasparente e amichevole, spiegando di essere un assistente AI basato su Rasa.

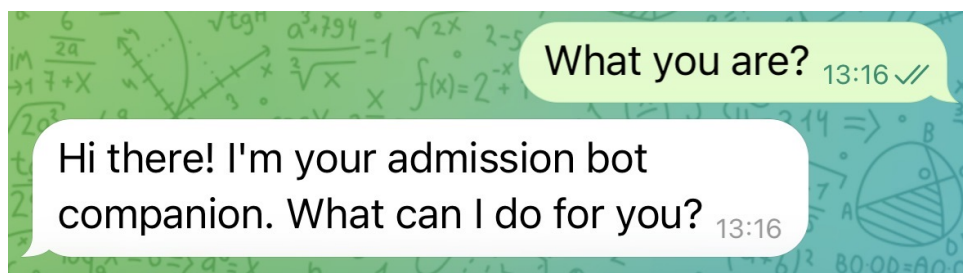


Figura 3.3: Esempio di risposta al bot challenge ("Sei un bot?").

3.1.3 FAQ: Scadenze, Costi e Requisiti

Il bot risponde automaticamente a tre categorie principali di domande frequenti:

Scadenze di Iscrizione (`ask_deadlines`)

Quando l'utente chiede "Quando scade il bando?" o "When is the deadline?", il bot attiva l'intent `ask_deadlines` e fornisce informazioni su:

- **Semestre autunnale:** scadenza 15 settembre.
- **Semestre primaverile:** scadenza 10 gennaio.

La risposta include entrambe le date, lasciando all'utente la scelta del semestre di interesse.

Un esempio di questa interazione è riportato nella seguente figura, dove l'utente chiede le scadenze di iscrizione e riceve una risposta dettagliata e chiara dal bot.

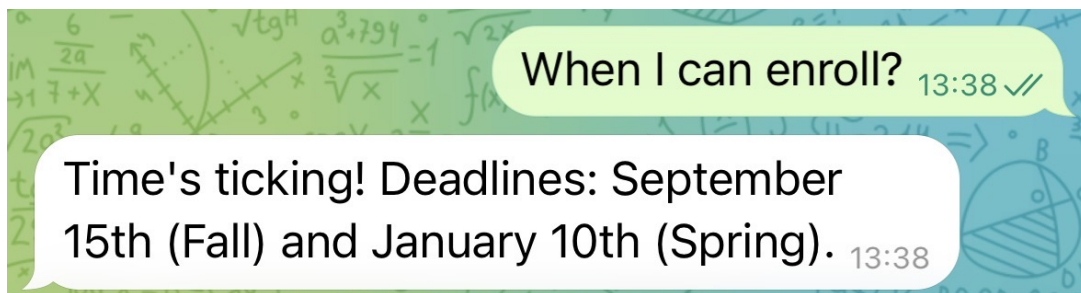


Figura 3.4: Esempio di richiesta informazioni sulle scadenze di iscrizione.

Costi e Tasse (ask_costs)

L'intent `ask_costs` fornisce informazioni sui costi di iscrizione:

- **Prima rata:** circa 156€ (importo fisso).
- **Rate successive:** variabili in base all'ISEE dello studente.

Il bot spiega che le tasse sono progressive e basate sulla situazione economica, incoraggiando l'utente a contattare l'ufficio ammissioni per dettagli specifici.

Un esempio di questa funzionalità è visibile di seguito, dove l'utente richiede informazioni sui costi e il bot fornisce una risposta chiara e professionale.

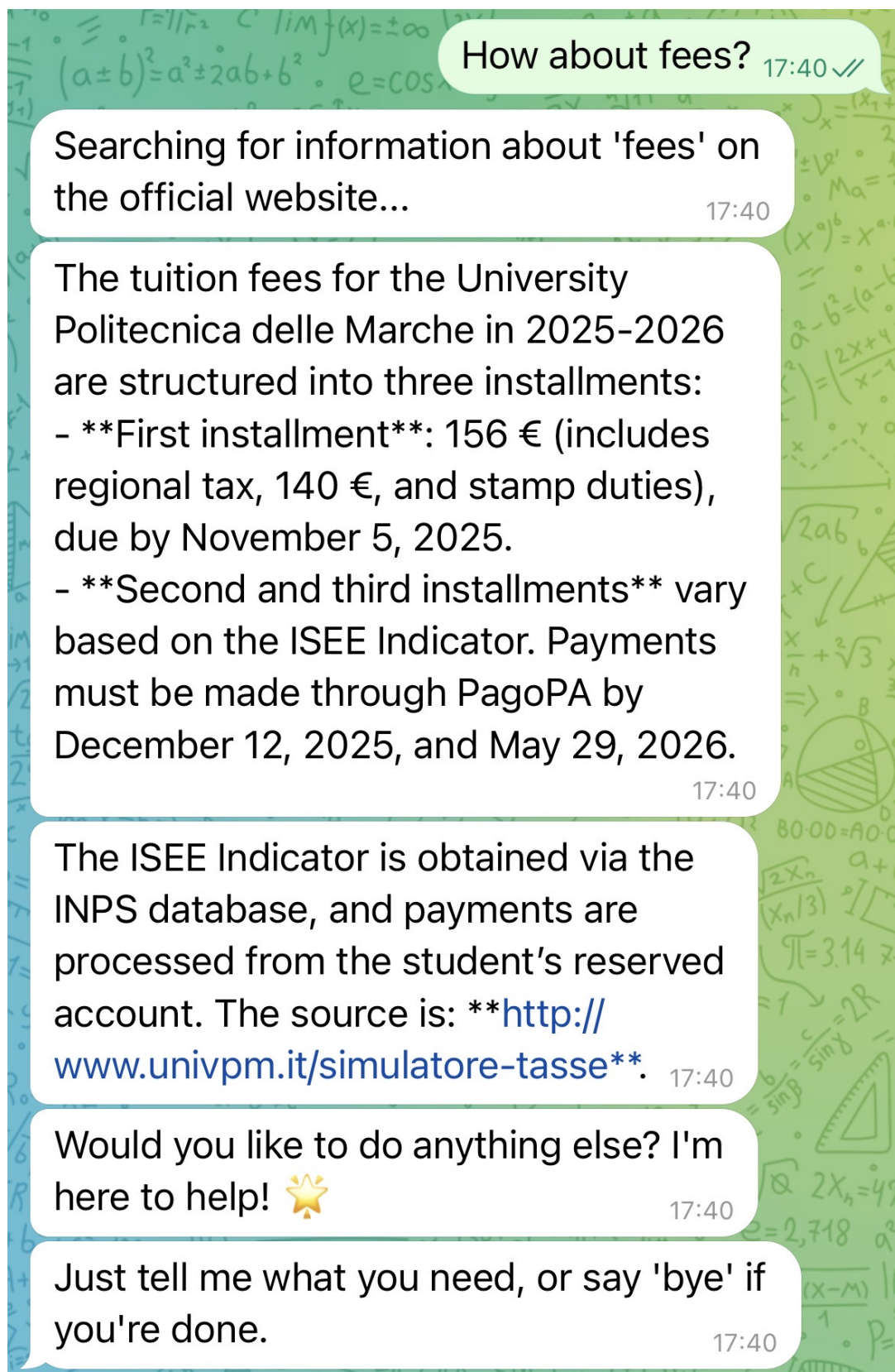


Figura 3.5: Esempio di richiesta informazioni sui costi universitari.

3.1.4 Form di Iscrizione (`enrollment_form`)

La funzionalità più sofisticata del bot è la form di iscrizione conversazionale, che guida lo studente attraverso un processo strutturato di raccolta dati.

Attivazione della Form

La form viene attivata quando l'utente esprime l'intento di iscriversi, tramite frasi come:

- "Voglio iscrivermi"
- "I want to enroll"
- "Apply for a course"

L'intent `apply_for_course` avvia il ciclo della form.

Raccolta dei dati - slot 1: nome studente

Il primo slot richiesto è `student_name`. Il bot chiede: "Qual è il tuo nome completo?"

La raccolta del nome utilizza un slot mapping avanzato:

- **from_entity:** se l'NLU estrae un'entità `student_name` dal testo (tramite entity extraction).
- **from_text:** se il form è attivo e in attesa del nome, accetta qualsiasi testo come nome (fallback).

Questo approccio è robusto: anche se l'NLU non riconosce il nome come entità, il bot lo accetta comunque come testo libero.

Un esempio di questa interazione è riportato nella figura seguente, dove l'utente fornisce il suo nome e il bot lo acquisisce e conferma prima di procedere allo slot successivo.

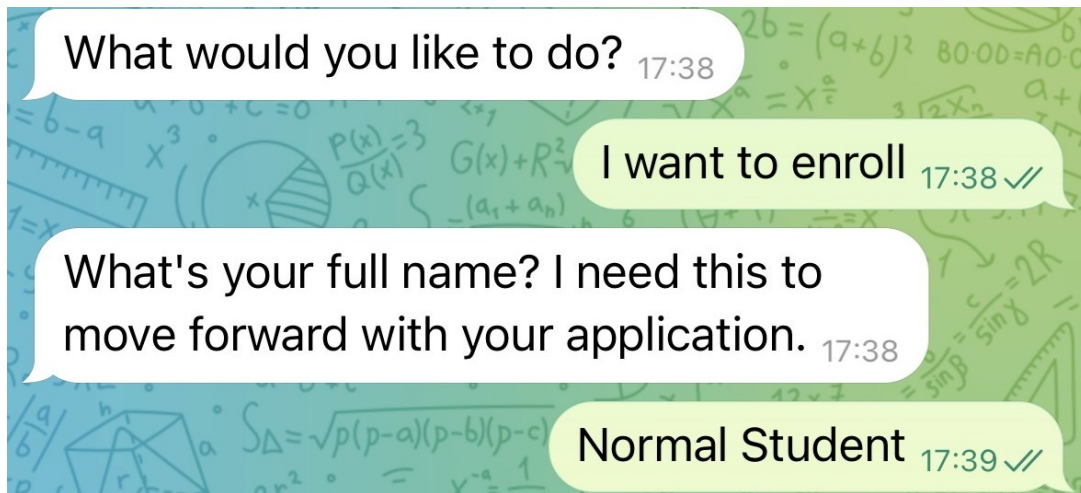


Figura 3.6: Esempio di raccolta del nome studente nel form di iscrizione.

Raccolta dei dati - slot 2: tipo di laurea

Il secondo slot è `degree_type`. Il bot chiede: "Desideri una laurea Triennale o Magistrale?"

L'utente risponde con l'entità `degree_type`, estratta dall'NLU. Il bot valida che il valore sia uno dei tipi riconosciuti (Triennale, Bachelor, Master, Magistrale, ecc.).

Un esempio di questa fase è mostrato di seguito, dove l'utente specifica il tipo di laurea desiderato e il bot conferma la selezione prima di procedere.

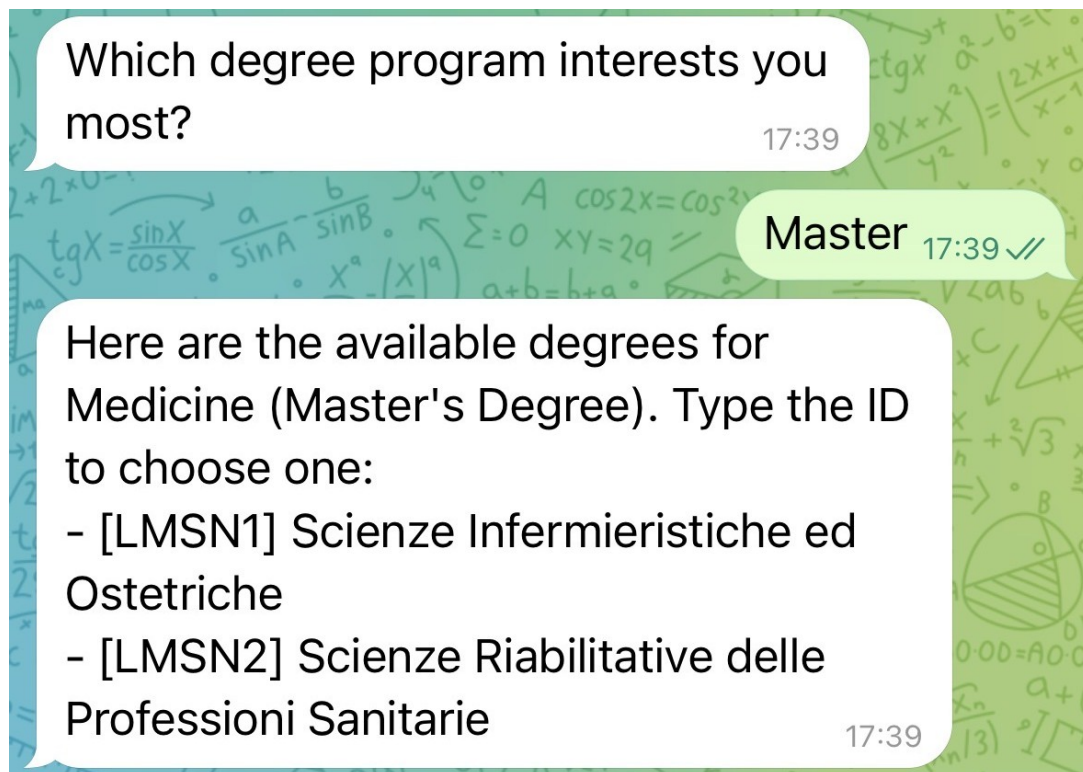


Figura 3.7: Esempio di selezione del tipo di laurea nel form di iscrizione.

Raccolta dei dati - slot 3: corso di studio

Il terzo slot è `course_name`. Il bot chiede: "Quale corso ti interessa? (es. Informatica, Medicina)"

L'utente fornisce il nome del corso, estratto come entità `course_name`.

Un esempio di questa fase è riportato di seguito, dove l'utente nomina il corso desiderato e il bot lo acquisisce, completando così tutti gli slot necessari.

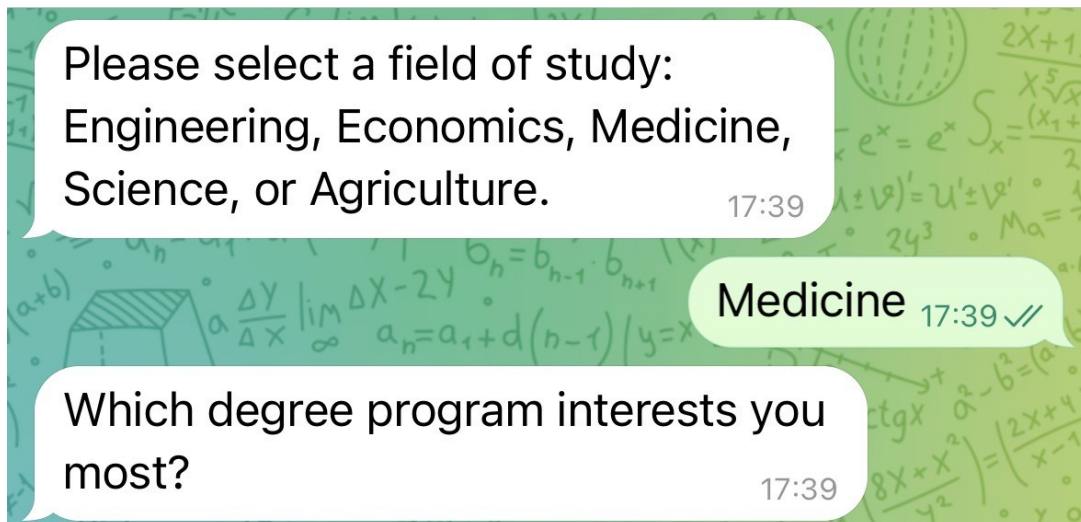


Figura 3.8: Esempio di selezione del corso di studio nel form di iscrizione.

Completamento e sottomissione della form

Una volta compilati tutti e quattro gli slot (nome, tipo laurea, corso, email), la form si completa e il bot attiva l'azione `utter_submit`, che conferma all'utente:

"Fantastico Mario! Ho iniziato la tua candidatura per una Magistrale in Informatica. Riceverai un'email a breve."

Questo messaggio è personalizzato con il nome dell'utente e i dettagli dell'iscrizione. La risposta varia ma è in inglese nel `domain.yml`.

Un esempio completo di questa fase conclusiva è illustrato di seguito, dove il bot conferma la ricezione di tutti i dati e personalizza la risposta con le informazioni fornite dall'utente.



Figura 3.9: Esempio di completamento e sottomissione del form di iscrizione con conferma personalizzata.

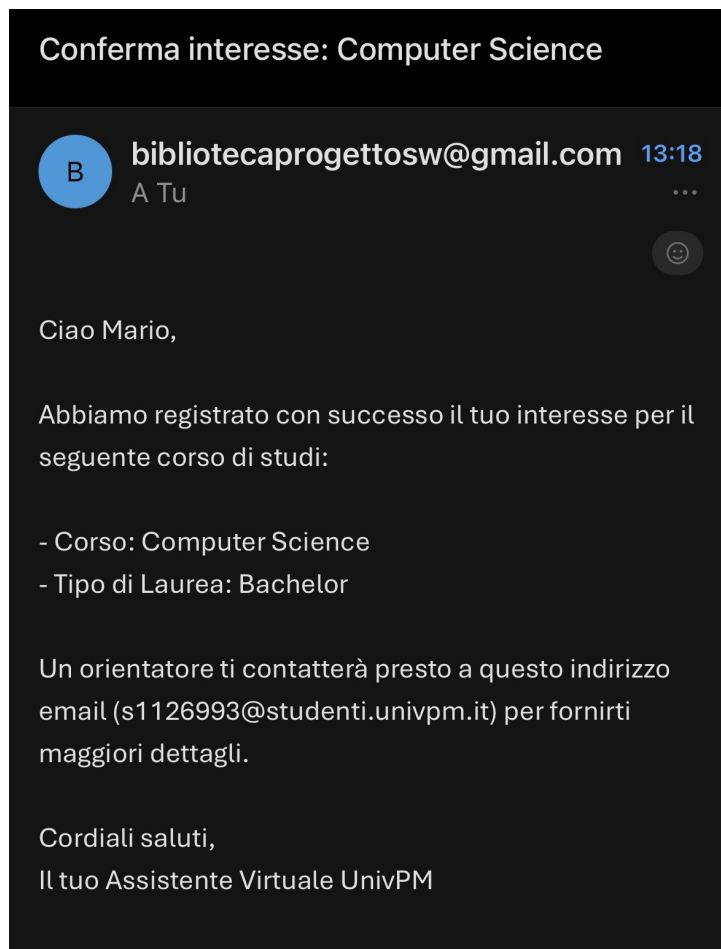


Figura 3.10: Email inviata dal bot.

Newsletter e raccolta email

Oltre al form di iscrizione, il bot supporta una funzionalità separata di newsletter attraverso l'intento `newsletter_request`. Quando l'utente chiede di iscriversi alla newsletter (frasi come "Iscrivimi alla newsletter" o "I want to subscribe to the newsletter"), il bot attiva il `newsletter_form`, che raccoglie solo l'email dell'utente.

Una volta raccolta l'email, viene automaticamente inviata una mail di conferma tramite `action_send_email`, e il bot risponde con `utter_email_sent` confermando la ricezione all'utente.

3.1.5 Error Recovery e Fallback

Il bot implementa meccanismi sofisticati di gestione degli errori:

FallbackClassifier

Il `FallbackClassifier` gestisce situazioni in cui la confidenza del sistema è bassa o quando due intent hanno score molto simili. In questi casi, il

bot risponde con l'intent `nlu_fallback` e chiede all'utente di riformulare la richiesta.

Un esempio di fallback è mostrato nella figura seguente, dove l'utente fornisce un input che il bot non riesce a riconoscere con sufficiente confidenza, e il bot chiede una riformulazione della richiesta.

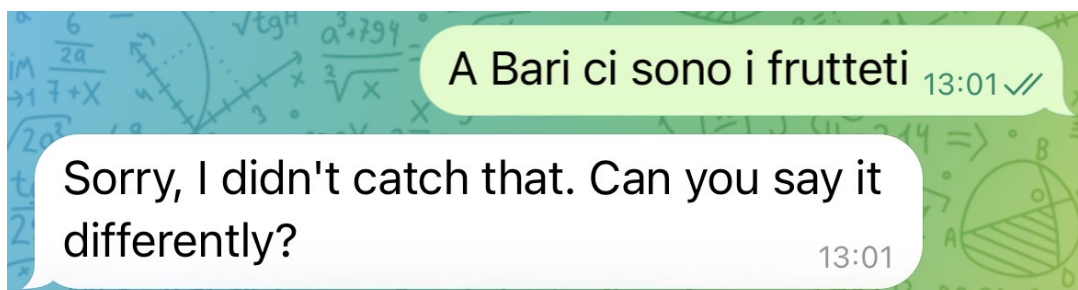


Figura 3.11: Esempio di attivazione del FallbackClassifier per richiesta non riconosciuta.

Out of Scope

Se l'utente pone domande completamente fuori ambito (ad es., "Che ora è?" o "Raccontami una barzelletta"), l'intent `nlu_fallback` o `out_of_scope` viene attivato, e il bot risponde:

"Mi scuso, ma non ho capito bene. Potresti riformulare?"

Questo mantiene il focus sulla missione del bot: assistere con l'ammissione universitaria.

Un esempio di questa situazione è illustrato di seguito, dove l'utente pone una domanda completamente fuori contesto e il bot rifiuta educatamente, riportando l'attenzione al suo scopo principale.

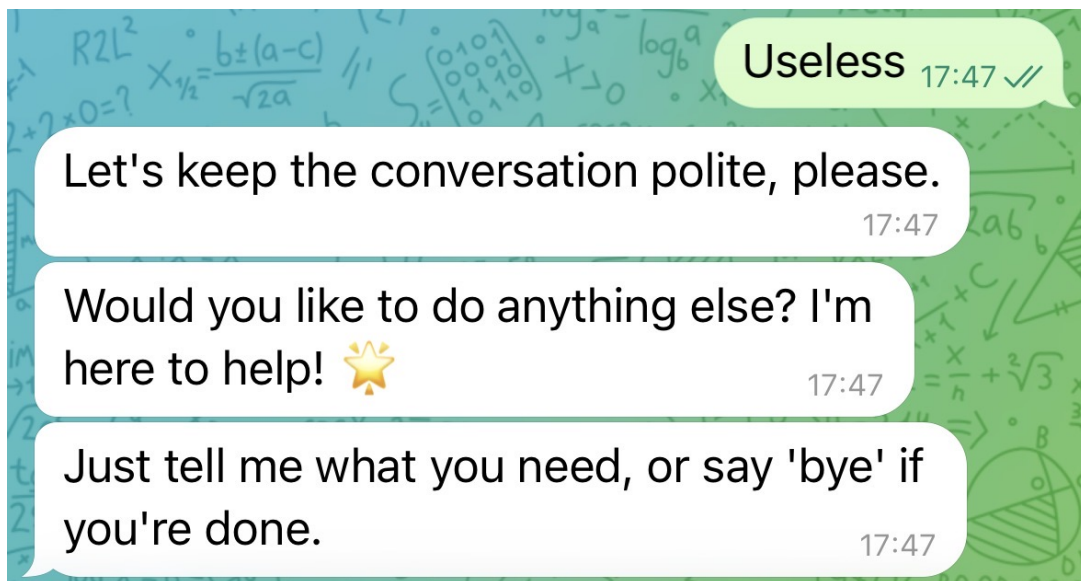


Figura 3.12: Esempio di gestione di domande fuori ambito con risposta appropriata.

3.2 Vantaggi e Limitazioni del Sistema

3.2.1 Vantaggi Principali

Il chatbot "Uncounsciously Sincere Bot" offre numerosi vantaggi rispetto ai sistemi tradizionali di informazione e iscrizione universitaria:

- **Disponibilità 24/7:** il bot è sempre disponibile, eliminando i tempi di attesa e permettendo agli studenti di ottenere informazioni in qualsiasi momento, indipendentemente dal fuso orario o dagli orari di ufficio.
- **Interazione Naturale:** grazie al NLU avanzato di Rasa, gli studenti possono porre domande in linguaggio naturale senza dover navigare menu complessi o seguire percorsi rigidi.
- **Supporto Multilingua:** il sistema bilingue permette di servire sia studenti italiani che internazionali.
- **Consistenza delle Risposte:** a differenza del supporto umano, il bot fornisce sempre risposte coerenti e accurate, basate sui dati configurati, eliminando il rischio di informazioni contraddittorie.
- **Scalabilità:** il bot può gestire un numero illimitato di conversazioni simultanee, rendendolo particolarmente utile durante i picchi di richieste (ad esempio, vicino alle scadenze di iscrizione).
- **Raccolta Dati Strutturata:** il form conversazionale raccoglie informazioni in modo standardizzato, facilitando l'integrazione con sistemi amministrativi universitari.

- **Tracciabilità:** tutte le conversazioni sono registrate, permettendo analisi per migliorare il servizio e identificare FAQ comuni non ancora coperte.

3.2.2 Limitazioni Attuali e Possibili Miglioramenti

Pur essendo un sistema funzionale e robusto, il bot presenta alcune limitazioni che potrebbero essere affrontate in future iterazioni:

- **Dominio Limitato:** il bot è specializzato nel processo di ammissione e non può rispondere a domande generali sull'università, la vita studentesca, o temi non correlati.
- **Manca di Integrazione Backend:** attualmente il bot non è connesso a database universitari reali. Le informazioni su scadenze e costi sono statiche e devono essere aggiornate manualmente nel codice.
- **Custom Actions Non Implementate:** il template per azioni personalizzate è presente ma non implementato. Funzionalità come l'invio automatico di email di conferma o la validazione dei corsi richiederebbero sviluppo aggiuntivo.
- **Gestione Limitata di Casi Edge:** situazioni particolari (studenti con crediti da trasferire, casi di ammissione speciale, ecc.) non sono attualmente gestite.

3.2.3 Possibili Estensioni Future

Il progetto è stato concepito con l'espandibilità in mente. Possibili estensioni includono:

1. **Integrazione con Database Universitari:** connettere il bot a sistemi di gestione universitari per fornire informazioni in tempo reale su disponibilità corsi, scadenze specifiche per programma, stato delle candidature.
2. **Implementazione Custom Actions:**
 - Generazione di numeri di candidatura univoci
 - Controllo automatico di requisiti basati su documenti caricati
 - Scheduling di appuntamenti con consulenti per l'orientamento
3. **Supporto Documenti:** permettere agli studenti di caricare documenti (diplomi, certificati) direttamente tramite Telegram e validarli automaticamente.

4. **Chatbot Vocale:** integrare il bot con assistenti vocali (Alexa, Google Assistant) per supportare studenti con disabilità visive.
5. **Analisi Sentimento:** implementare analisi del sentimento per identificare studenti frustrati e escalare automaticamente a supporto umano.
6. **Espansione Linguistica:** aggiungere supporto per altre lingue (spagnolo, francese, tedesco) per attrarre studenti internazionali da più paesi.
7. **Dashboard Amministrativa:** creare un'interfaccia web per lo staff universitario per monitorare conversazioni, aggiornare FAQ, e analizzare statistiche di utilizzo.
8. **Machine Learning Continuo:** implementare un sistema di raccolta feedback e re-training periodico per migliorare costantemente il modello.

Queste estensioni trasformerebbero il bot da un sistema di informazione e raccolta dati a un vero e proprio assistente amministrativo completo, capace di gestire l'intero ciclo di vita del processo di ammissione universitaria.

4 Esecuzione del chatbot

Il seguente capitolo illustra le procedure necessarie per addestrare il modello "Uncounsciously Sincere Bot", eseguirlo in locale e connetterlo alla piattaforma Telegram. Data la complessità delle dipendenze software, il progetto privilegia l'utilizzo di Docker per garantire la consistenza dell'ambiente su diverse macchine, sebbene venga trattata anche l'esecuzione tramite ambiente virtuale Python standard.

4.1 Setup iniziale e installazione

Prima di procedere con le fasi di training ed esecuzione, è indispensabile preparare l'ambiente di lavoro soddisfacendo alcuni requisiti preliminari e ottenendo il codice sorgente.

4.1.1 Requisiti di sistema

Per operare correttamente con il progetto sono necessari i seguenti strumenti:

- Docker Desktop (versione 20.10 o superiore).
- Docker Compose.
- Git per la gestione del repository.
- Un token Telegram ottenuto tramite BotFather.
- Un account Ngrok per il tunneling HTTPS.

4.1.2 Clonazione e configurazione base

Il primo passo consiste nell'ottenere il codice sorgente clonando il repository ufficiale ed entrando nella directory di progetto:

```
git clone https://github.com/DataScience-Golddiggers/  
Uncounsciously-Sincere-Bot.git  
cd Uncounsciously-Sincere-Bot
```

Successivamente è necessario configurare le credenziali. Nel progetto è presente un file template che deve essere duplicato e rinominato:

```
cp credentials.example.yml credentials.yml
```

All'interno del file `credentials.yml` dovrà essere inserito il token di Telegram. Il campo relativo all'URL del webhook verrà compilato in un secondo momento, dopo l'avvio del tunnel Ngrok.

4.2 Esecuzione tramite Docker Compose

L'architettura del bot si basa sull'orchestrazione di tre servizi principali gestiti tramite Docker Compose: il server Rasa, l'Action Server e Ngrok.

4.2.1 Configurazione del file docker-compose

Il file `docker-compose.yml` definisce le relazioni tra i container e le variabili d'ambiente. Una configurazione tipica per questo progetto appare come segue:

```
version: '3.8'

services:
  rasa:
    build: .
    ports:
      - "5005:5005"
    environment:
      - SMTP_SERVER=smtp.gmail.com
      - SMTP_PORT=465
      - SMTP_EMAIL=${SMTP_EMAIL}
      - SMTP_PASSWORD=${SMTP_PASSWORD}
    volumes:
      - ./models:/app/models
      - ./data:/app/data

  action_server:
    build: ./actions
    ports:
      - "5055:5055"
    environment:
      - SMTP_SERVER=smtp.gmail.com
      - SMTP_PORT=465
      - SMTP_EMAIL=${SMTP_EMAIL}
```

```
- SMTP_PASSWORD=${SMTP_PASSWORD}
```

```
ngrok:  
  image: ngrok/ngrok:latest  
  ports:  
    - "4040:4040"  
  environment:  
    - NGROK_AUTHTOKEN=${NGROK_AUTHTOKEN}  
  command: http rasa:5005
```

Questa configurazione espone il server Rasa sulla porta 5005, l'Action Server sulla 5055 e avvia Ngrok per creare un tunnel sicuro verso il servizio Rasa.

4.2.2 Costruzione dell'immagine e avvio

Il Dockerfile presente nella radice del progetto si occupa di installare le dipendenze Python e la versione specifica di Rasa (3.6.13) necessaria per la compatibilità. Per avviare l'intera infrastruttura in modalità distaccata (background), si utilizza il comando:

```
docker-compose up -d
```

Una volta lanciato il comando, i servizi si avvieranno in parallelo. Per arrestare l'esecuzione e liberare le risorse, il comando da utilizzare è:

```
docker-compose down
```

4.2.3 Recupero dell'indirizzo Ngrok

Affinché Telegram possa comunicare con il bot in esecuzione sulla macchina locale, è necessario fornire un indirizzo HTTPS pubblico. Ngrok svolge questa funzione. Una volta avviati i container, è possibile recuperare l'URL pubblico accedendo alla dashboard locale di Ngrok tramite browser:

```
http://localhost:4040
```

Dalla dashboard si potrà copiare l'URL generato (solitamente terminante in `.ngrok.io`) che dovrà essere inserito nel file `credentials.yml`.

4.3 Esecuzione in ambiente locale alternativo

Se si preferisce non utilizzare Docker, è possibile eseguire il bot tramite un ambiente virtuale Python tradizionale.

4.3.1 Installazione delle dipendenze

È necessario creare un ambiente virtuale con Python 3.10 e installare i pacchetti richiesti:

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Questo installerà Rasa, l'SDK per le azioni e le librerie necessarie per la gestione delle email.

4.3.2 Training ed esecuzione manuale

A differenza di Docker, dove i processi sono automatizzati, qui è necessario procedere manualmente. Il primo passo è l'addestramento del modello:

```
rasa train
```

Al termine del training, che genererà un file compresso nella cartella `models`, occorre aprire due terminali distinti per eseguire i servizi.

Nel primo terminale si avvia il server delle azioni:

```
rasa run actions
```

Nel secondo terminale si avvia il server Rasa abilitando le API per la comunicazione esterna:

```
rasa run --enable-api
```

Se si desidera testare il bot senza interfacce esterne come Telegram, è possibile sostituire l'ultimo comando con `rasa shell` per interagire direttamente da riga di comando.

4.4 Configurazione delle email

Il bot include una funzionalità per l'invio automatico di email di conferma. Per abilitarla, occorre configurare le credenziali SMTP.

4.4.1 Configurazione per Gmail

L'utilizzo di Gmail richiede una procedura di sicurezza specifica poiché la password standard dell'account non è sufficiente per l'accesso da applicazioni terze:

1. Accedere al proprio account Google e navigare nella sezione Sicurezza.

2. Attivare la verifica in due passaggi se non presente.
3. Generare una "Password per le app" selezionando "Posta" come app e il proprio dispositivo.
4. Copiare la password di 16 caratteri generata.

4.4.2 Variabili d'ambiente

Le credenziali ottenute devono essere salvate in un file `.env` nella root del progetto, strutturato come segue:

```
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=465
SMTP_EMAIL=tua-email@gmail.com
SMTP_PASSWORD=tua-password-app
```

Queste variabili vengono lette automaticamente sia nell'esecuzione locale che tramite Docker Compose.

4.5 Integrazione con Telegram

Rasa supporta nativamente l'integrazione con Telegram, ma richiede una configurazione precisa del webhook.

4.5.1 Creazione del bot e token

La procedura inizia su Telegram:

1. Cercare l'utente "BotFather" e avviare la chat.
2. Inviare il comando `/newbot`.
3. Seguire le istruzioni per assegnare nome e username al bot.
4. Copiare il token di accesso fornito al termine della procedura.

4.5.2 Modifica delle credenziali

Il token va inserito nel file `credentials.yml`. È inoltre fondamentale specificare il webhook URL corretto, composto dall'indirizzo Ngrok e dal percorso API di Rasa:

```
telegram:
  access_token: "IL_TUO_TOKEN_QUI"
  verify: "IL_TUO_USERNAME"
  webhook_url: "https://XXXX.ngrok.io/webhooks/telegram/webhook"
```

Dopo aver salvato il file, è necessario riavviare il server Rasa affinché le modifiche abbiano effetto.

4.6 Risoluzione dei problemi

Durante le fasi di setup ed esecuzione possono verificarsi errori comuni. Di seguito vengono analizzate le casistiche più frequenti.

4.6.1 Mancata risposta su Telegram

Se il bot non risponde ai messaggi, le cause principali sono solitamente due:

- Il tunnel Ngrok è scaduto o è stato riavviato, cambiando l'URL pubblico. In questo caso è necessario aggiornare il file `credentials.yml` con il nuovo indirizzo.
- Il token di Telegram è errato. Verificare la corrispondenza esatta con quanto fornito da BotFather.

4.6.2 Porta 5005 già in uso

L'errore "Address already in use" indica che un altro processo sta occupando la porta necessaria a Rasa. Spesso accade quando un container Docker non è stato chiuso correttamente. Per risolvere, assicurarsi di aver eseguito `docker-compose down` o terminare manualmente i processi python attivi.

4.6.3 Errori di memoria (OOM)

Durante il training, specialmente su macchine con risorse limitate, il processo potrebbe terminare per mancanza di memoria. Le soluzioni possibili includono l'aumento della RAM allocata a Docker o la riduzione del numero di epoche nel file di configurazione `config.yml`.

4.7 Considerazioni per la produzione

L'ambiente descritto è ottimizzato per lo sviluppo. Per un rilascio in produzione, è necessario adottare ulteriori misure di sicurezza e stabilità.

La gestione delle credenziali deve essere rigorosa: file contenenti token o password non devono mai essere esposti pubblicamente. Inoltre, l'utilizzo di Ngrok dovrebbe essere sostituito da un dominio proprietario con certificati SSL gestiti tramite un reverse proxy come Nginx.

Per garantire la persistenza delle conversazioni anche in caso di riavvio del server, è consigliabile configurare un Tracker Store su database stabile (ad esempio PostgreSQL) al posto della memoria volatile utilizzata di default durante lo sviluppo. Infine, implementare un sistema di logging e monitoraggio permette di analizzare le interazioni degli utenti e migliorare progressivamente le capacità del modello.